

Part 1 — Positioning, and the app as a thing a human holds

Format for every question:

- **Q** — how a sharp panel member would actually phrase it
 - **Think** — the chain of reasoning, out loud, before the answer
 - **Today** — what the code does on 2026-09-03, cited
 - **At scale** — what changes, and what it costs
 - **Breaks when** — the honest failure condition
-

Q1.1 — "IndiaHandmade already exists. Government-built, government-run, D2C, for verified weavers. Why does your app exist?"

Think. This is not a hostile question, it is the best question, and the answer is a number they may not know. If I argue "ours is better designed" I lose — the government built theirs and one of the judges may have signed off on it. The only survivable answer treats IndiaHandmade as *evidence*, not competition.

Answer. IndiaHandmade onboarded **4,186 weavers and artisans between 2023–24 and June 2026**. Against roughly 35 lakh handloom workers that is **≈0.1% penetration in three years**.

That is not a failure of the storefront. It is proof that **the storefront was never the bottleneck**. The government built the shop and the shelves stayed empty, because getting an artisan's product *ready to be listed* — a white-background photograph, a category, an attribute schema, a description in two languages, a defensible price — is the actual work, and nobody automated it.

We are not building another storefront. We are building the onboarding layer that fills every storefront, including that one.

Today. Our marketplace exists (`web/api/routers/marketplace.py`) but is deliberately *not* the pitch — `docs/Master-Technical-Reference.md` §3.3 justifies it as the B2B surface and the zero-paperwork fallback, and as the only channel we can demonstrate end to end on a stage.

At scale. The correct integration is that we push **into** IndiaHandmade too. It is another adapter in `web/api/channels/` — `map_category`, `map_attributes`, `format_images`, `render`. Adding a channel never touches core (`docs/Master-Technical-Reference.md` §8.8).

Breaks when. If IndiaHandmade's 0.1% is caused by *demand*, not supply-side friction, we have automated the wrong half. See Q9.2 — we do not fully know, and we say so.

Q1.2 — "You have built a supply-side tool. Nothing in it creates a buyer. What happens to the artisan who lists twenty products and sells nothing?"

Think. This is the question I would ask if I wanted to end a pitch. There is no clever answer. The honest answer is to concede the frame and then show the two places we do touch demand.

Answer — the concession first. Correct. Cataloging is necessary and not sufficient. An artisan with a perfect listing and no traffic has a prettier version of the same problem, and a system that raised their hope without raising their income has cost them something real.

The three places we do touch demand, in order of honesty:

1. **GeM's MSE quota.** A minimum of **25% of government procurement is reserved for MSMEs**. This is guaranteed demand that exists whether or not we are clever — and it is procedurally gated, not marketing gated. The barrier is a correctly filled catalogue, which is exactly the thing we automate. This is the strongest demand answer we have and it is the reason GeM is our primary channel and not Amazon.
2. **ONDC fan-out.** One catalog entry becomes visible across every buyer app on the network simultaneously. We do not create demand; we stop the artisan from being invisible to it.
3. **Bulk RFQ on our marketplace** (`/rfq/:productId`). A shopkeeper who needs 200 gamchas has no way to express that on Amazon. That flow is genuinely a direct artisan-to-buyer connection, which is the PS's literal wording.

Breaks when. All three are channels, not customers. If GeM buyers do not search handloom, if ONDC buyer-app traffic stays low, if no B2B buyer visits our marketplace — the artisan lists into silence. **We measure this and we would rather know:** Order rows per artisan per channel is the metric that tells us whether the thesis holds, and it is in the schema from day one (`docs/Master-Technical-Reference.md §15`).

Q1.3 — "Your problem statement says cross-platform mobile app. You built React in a WebView. Why is that not a shortcut?"

Think. The judge is testing whether we chose Capacitor for convenience or for a reason. The reason has to be about the user's phone, not our comfort.

Today. React + Vite + Capacitor, Android first (`docs/Application-Architecture.md §0.6`). The honest driver: one codebase, and the app is a thin client. **Every AI feature is a server call anyway** — enhancement, ASR, description, pricing, publish. There is no heavy native computation to justify two native codebases.

The one place the WebView genuinely costs us, and we measured it: getting camera frames into JavaScript means `video → canvas → getImageData`, and that readback is the expensive step. At full resolution we would get ~2 fps and the live camera gate would be worthless.

That is why the gate samples at **240×180 grayscale** (`docs/Master-Technical-Reference.md §4.3`) — 43,000

pixels instead of 12 million, one buffer read by all four checks, 10–15 checks/sec, which reads as "live" to the eye. Raising that sample size is on the **rejected** list in `CLAUDE.md`, because it is the single change that kills the feature.

At scale. If frame throughput ever becomes the constraint, the fix is a small native Android plugin that processes frames on the native side — a bounded piece of work behind an interface that already exists (app/src/camera/gate.js is pure arithmetic, no DOM, and runs under plain node, 19 assertions, no device or framework).

Breaks when. A ₹7,000 phone with 2–3 GB RAM under memory pressure — the WebView is the first thing Android reclaims. We have not measured this on a genuinely low-end device, and that is an open gap, not a solved one.

Q1.4 — "You call it voice-first. Show me the one thing the artisan still has to type."

Think. Answer the literal question first — they are checking whether the claim is absolute or marketing.

Today. The OTP. That is the entire list (docs/Application-Architecture.md §3 rule 3, and the onboarding flow, §4). Phone + OTP is the account: no password, no email, no username. The remaining seven onboarding screens are language tiles, a spoken consent notice, a spoken name, an eight-icon craft grid, a pincode, and four spoken yes/no questions.

Onboarding is **eight screens, roughly ninety seconds, and it ends inside the camera** — no product tour, no dashboard. Nobody who cannot read wants a product tour.

The design law that enforces it (§3): ≤ 3 tappable things per screen; every screen speaks on entry; nothing typed except the OTP; one problem shown at a time. A screen that breaks one is rejected in review.

At scale. Android SMS Retriever API removes even the OTP typing. Cheap, and it takes the "except the OTP" clause out of the sentence entirely.

Breaks when. The OTP does not arrive — rural tower, DND registry, operator throttling. There is no fallback path today. That is a real hole for exactly our user, and a missed-call-verification or IVR fallback is the fix.

Q1.5 — "Voice-first is useless in a noisy weaving shed. Have you been in one?"

Think. Do not bluff field experience we do not have. Answer with architecture instead, because the architecture happens to already handle it.

Today. Every voice path has a non-voice fallback that is not a degraded mode but the primary UI for that screen:

Screen	If voice fails
/onboard/ craft	Eight-icon grid. Voice is the "something else" escape hatch, not the default
/camera	Nothing is spoken to make the gate work. Colour ring, large tick/cross icon, haptic buzz, and the shutter physically greys out until the shot is good
/catalog/ voice	Each of the six questions is individually skippable
Confirmations	<code>classifyYesNo()</code> returns <code>null</code> rather than guessing, and the caller re-asks

The camera answer is the important one. docs/Master-Technical-Reference.md §4.7: red state → shutter disabled and grey; green → green and glowing; hold green one second → auto-capture. **The artisan does not have to understand anything. They move until the button lights up.** That is zero literacy *and* zero audio.

At scale. Noise-robustness is an ASR model property, and it is the strongest single argument for Bhashini over a general commercial API — Indian-language, Indian-acoustics training data. Unverified (docs/decisions.md #5). We would want a field recording set from an actual shed before claiming anything.

Breaks when. The description path genuinely needs speech. If ASR word-error-rate is bad in noise, `compose_fallback` builds the listing from the artisan's own raw answers and marks it `confidence: 0` — the listing survives, the polish does not. **An unreachable model never raises** (CLAUDE.md, Current state).

Q1.6 — "One phone per household, shared between four adults. Your account is a phone number. Whose products are these?"

Think. This is a real field condition and our data model has a real answer only halfway.

Today. Artisan is keyed on phone (docs/Master-Technical-Reference.md §15). One phone = one artisan account. Products belong to the artisan id, and payouts go to a UPI id stored for that account.

The honest gap. If a mother and daughter both weave and share one handset, they share one account, one catalog, and one UPI id. Attribution — and therefore *income attribution*, which is the PS's stated impact goal — collapses into a household rather than a person. For a scheme targeting women artisans specifically (~16.87 lakh registered with DC-Handicrafts), that is not a cosmetic loss.

At scale. The fix is a profile switcher under one device session, not separate logins: the account holds N artisan profiles, the camera and catalog scope to the selected one, and the selection is a face-sized tile with a spoken name, not a dropdown. Schema change is a nullable `profile_id` on `Product` and `Order`. It is not built, and it should be before any real pilot.

Breaks when. Today, right now, in every shared-phone household. We do not currently even *detect* the case.

Q1.7 — "Your own README says the artisan may be on a rural tower. Then you chose online-first. Explain that."

Think. This looks like a contradiction and it is the decision I most expect to be attacked on. The defence has to be arithmetic, not preference.

Today. Online-first, decided deliberately and recorded as a reversal (`docs/decisions.md`; `docs/Master-Technical-Reference.md` §3.1 carries the amendment banner).

The reasoning, in one line: every AI feature in this app is a server call. Enhancement, ASR, description, pricing, publish — all of them. **Offline capture without offline inference gets the artisan a photograph and nothing else.** They cannot see the enhanced image, cannot get a description, cannot get a price, cannot publish. They get a photo, which their camera app already gave them.

Against that, offline-first costs an offline queue, a local database and a sync engine — a multi-week subsystem with a class of merge-conflict bugs we cannot afford, and nothing at the end of it that demonstrates.

What we built instead, and it is the whole network story: resumable chunked upload with retry and backoff (`app/src/api/upload.ts`, `app/src/api/resume.js`), and **spoken failure** — *"network nahi hai, thodi der me dobara"* — never a silent spinner. Rule 3 in `CLAUDE.md`: every failure degrades and speaks.

And one part genuinely is offline. `stripCarrier()` and `matchCraft()` in `app/src/voice/interpret.ts` run **before** the model, on-device, in three languages, and their answer is used immediately when confident. `docs/app/Future-Implementations.md` §3 says explicitly not to delete them when the model gets good: *"They are what makes the app work on a phone with no signal, which is where our users actually are."* The common case sends **nothing at all** to any model.

At scale. Revisit only if field testing proves connectivity is the blocker — that is written into the decision, not added under questioning.

Breaks when. Intermittent connectivity that is good enough to start an upload and not good enough to finish it, repeatedly. Resumable upload handles a drop; it does not handle a tower that gives 20 kbps for an hour. A 4 MB photo at 20 kbps is 27 minutes. We have no client-side downscale-before-upload path today, and that is the cheapest fix available to us.

Q1.8 — "How large is the app? Our users are on 2 GB phones with 8 GB of storage and they pay for data by the megabyte."

Think. Give the architecture-level answer, then concede the measurement we do not have.

Today. The app carries **no model**. On-device segmentation (U²-Netp), OpenCV.js and ONNX Runtime Web are all

on the **rejected** list in `CLAUDE.md` — one model, server-side. The camera gate is arithmetic. What ships is React + Tailwind + the Capacitor shell.

The thing that is not in the bundle and should be noticed: `ai/thresholds.json` is **fetches at runtime**, with the bundled copy acting only as a build-time floor (`app/src/api/client.js:185`). One file, two readers, never a second copy, never hardcoded. A recalibrated threshold reaches a rural phone as a config fetch, **not as an app update those users will never install**.

The same pattern governs the autofill selector packs (`docs/Application-Architecture.md §6.4`): versioned JSON from our server, never compiled in. *"A DOM change is a config push we ship in an hour — not an app update rural users will never install. This single choice is the difference between a feature that survives and one that silently rots."*

Honest gap. We have not measured the installed APK size or the per-listing data cost on a real handset. Both are one afternoon of work and neither is done. **The per-listing data cost is the one that matters more**, because the artisan pays it: one 4 MB upload per photo today, with no pre-upload downscale.

At scale. Downscale to the 2000 px master **on the phone before upload** — the server immediately reduces to that anyway (`to_master()`, 12 MP → 2000 px). Uploading 12 MP so the server can throw away 80% of it is the artisan's money spent on nothing. This is the single highest-value unbuilt item in the app.

Q1.9 — "Twenty-five screens. You told me three taps per screen and no reading. Which screen is the one that fails that test?"

Think. Naming our own worst screen is more convincing than defending all of them.

Answer. `/publish` (`docs/Application-Architecture.md §5`, route 15). It shows one large button plus a six-row channel status list — six rows with three distinct states each (`turant`, `file taiyar hai`, `jodna hai`, `madad chahiye`). That is more information than any other screen in the app and it is aimed at a user who cannot read the row labels.

Why we kept it. The big button is ~30% of the screen and fires everything that can fire without the artisan understanding a single row. The rows are *status*, not *choices* — they are read by the cluster coordinator over a phone call, and by us in a demo. Tier C and D become follow-up cards afterwards; they never block the tap.

The remaining fix, unbuilt: the six rows should be spoken as a summary on entry — *"do jagah bhej diya, GeM ki file taiyar hai"* — not left as six lines of text with icons. Until that ships, `/publish` is the screen that most depends on a sighted, literate helper.

Second-worst: `/earnings`. Money is the one place where a wrong spoken number destroys trust permanently, so it says *"Amazon ne ₹4,200 bheje hain — 15 tareekh tak aane chahiye"* and never *"₹4,200 aa gaye"* (`docs/Master-Technical-Reference.md §11.2`). That distinction is a whole sentence of nuance delivered by voice, and it is the hardest thing in the app to say simply.